

一、系统环境准备（所有节点执行）

1. 关闭Swap分区（必须）

Kubernetes要求永久关闭Swap，否则kubelet无法正常启动。

```
1 # 临时关闭Swap
2 sudo swapoff -a
3
4 # 永久关闭Swap（注释掉fstab中的swap行）
5 sudo sed -i '/swap/s/^/#/' /etc/fstab
6
7 # 验证Swap已关闭（输出应为空）
8 cat /etc/fstab | grep swap
9 free -h
```

2. 关闭防火墙

```
1 sudo ufw disable
2 sudo systemctl stop ufw
3 sudo systemctl disable ufw
```

3. 加载内核模块并配置系统参数

```
1 # 加载必要的内核模块
2 sudo modprobe overlay
3 sudo modprobe br_netfilter
4
5 # 配置sysctl参数
6 cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
7 net.bridge.bridge-nf-call-iptables = 1
8 net.bridge.bridge-nf-call-ip6tables = 1
9 net.ipv4.ip_forward = 1
10 EOF
```

```
11
12 # 应用参数
13 sudo systemctl --system
```

4. 设置主机名和hosts解析

```
1 # 设置主机名（替换为你的主机名）
2 sudo hostnamectl set-hostname k8s-master
3
4 # 配置hosts文件（添加本机IP和主机名映射，请将192.168.1.100替换为你服务器的实际内网IP地址）
5 echo "192.168.1.100 k8s-master" | sudo tee -a /etc/hosts
```

二、安装Containerd容器运行时

Kubernetes 1.24+不再默认使用Docker作为容器运行时，推荐使用Containerd。

1. 安装依赖

```
1 sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release
```

2. 添加Docker官方GPG密钥和apt源

```
1 sudo mkdir -p /etc/apt/keyrings
2 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
  /etc/apt/keyrings/docker.gpg
3
4 echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
  /etc/apt/sources.list.d/docker.list > /dev/null
```

3. 安装Containerd

```
1 sudo apt update
```

```
2 sudo apt install -y containerd.io
```

4. 配置Containerd

```
1 # 生成默认配置文件
2 sudo containerd config default | sudo tee /etc/containerd/config.toml
3
4 # 修改配置，使用systemd作为cgroup驱动（必须）
5 sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
6
7 # 重启Containerd并设置开机自启
8 sudo systemctl restart containerd
9 sudo systemctl enable containerd
10
11 # 验证Containerd状态
12 sudo systemctl status containerd
```

三、安装Kubernetes 1.28组件

1. 添加Kubernetes官方apt源

```
1 # 添加Google Cloud公钥
2 curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo gpg --dearmor -
o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
3
4 # 添加Kubernetes 1.28版本源
5 echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
```

2. 安装指定版本的kubeadm、kubelet、kubectl

```
1 sudo apt update
2 sudo apt install -y kubelet=1.28.10-1.1 kubeadm=1.28.10-1.1 kubectl=1.28.10-1.1
3
```

```
4 # 锁定版本，防止自动更新
5 sudo apt-mark hold kubelet kubeadm kubectl
6
7 # 验证安装
8 kubeadm version
9 kubectl version --client
```

四、初始化Kubernetes Master节点

1. 拉取所需镜像（可选，提前拉取加快初始化速度）

```
1 sudo kubeadm config images pull --kubernetes-version=1.28.10
```

2. 执行初始化命令

```
1 sudo kubeadm init \
2   --kubernetes-version=1.28.10 \
3   --apiserver-advertise-address=192.168.1.100 \
4   --pod-network-cidr=10.244.0.0/16 \
5   --service-cidr=10.96.0.0/12 \
6   --cri-socket=unix:///var/run/containerd/containerd.sock
```

参数说明：

- `--apiserver-advertise-address`：替换为你的服务器内网IP
- `--pod-network-cidr`：Pod网络段，后续安装Calico网络插件需要使用这个网段
- `--service-cidr`：Service网络段

3. 配置kubectl访问权限

```
1 mkdir -p $HOME/.kube
2 sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
3 sudo chown $(id -u):$(id -g) $HOME/.kube/config
4
5 # 验证kubectl连接
```

```
6 kubectl get nodes
```

此时你会看到Master节点状态为 `NotReady`，这是因为还没有安装网络插件。

五、安装Calico网络插件

Calico是Kubernetes最常用的网络插件之一，性能稳定且功能丰富。

```
1 # 下载Calico 3.26版本（兼容K8s 1.28）
2 kubectl apply -f
  https://raw.githubusercontent.com/projectcalico/calico/v3.26.4/manifests/calico.yaml
3
4 # 等待Calico Pod全部就绪（大约需要1-2分钟）
5 kubectl wait --for=condition=ready pod -l k8s-app=calico-node -n kube-system --
  timeout=300s
6 kubectl wait --for=condition=ready pod -l k8s-app=calico-kube-controllers -n kube-system
  --timeout=300s
7
8 # 再次验证节点状态（此时应该变为Ready）
9 kubectl get nodes
```

六、移除Master节点污点（单节点部署必须）

默认情况下，Kubernetes不会在Master节点上调度Pod。单节点部署需要移除这个污点：

```
1 kubectl taint nodes k8s-master node-role.kubernetes.io/control-plane-
```

输出 `node/k8s-master untainted` 表示成功。

七、部署Kubernetes Dashboard可视化界面

1. 部署Dashboard v2.7.0（兼容K8s 1.28）

```
1 #这步网络问题会失败
2 kubectl apply -f
  https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
3
4 #创建本地yaml文件
```

```
5 vim dashboard-aliyun.yaml
```

2. 粘贴以下完整内容到dashboard-aliyun.yam（已替换所有镜像为阿里云）

```
1 # Copyright 2017 The Kubernetes Authors.
2 # Licensed under the Apache License, Version 2.0
3 # 已将所有镜像替换为阿里云国内镜像: registry.cn-hangzhou.aliyuncs.com/google_containers
4
5 apiVersion: v1
6 kind: Namespace
7 metadata:
8   name: kubernetes-dashboard
9
10 ---
11 apiVersion: v1
12 kind: ServiceAccount
13 metadata:
14   labels:
15     k8s-app: kubernetes-dashboard
16   name: kubernetes-dashboard
17   namespace: kubernetes-dashboard
18
19 ---
20 kind: Service
21 apiVersion: v1
22 metadata:
23   labels:
24     k8s-app: kubernetes-dashboard
25   name: kubernetes-dashboard
26   namespace: kubernetes-dashboard
27 spec:
28   ports:
29     - port: 443
30       targetPort: 8443
31       nodePort: 30000 # 已提前设置NodePort端口, 无需后续修改
32   type: NodePort # 已改为NodePort, 直接通过IP:30000访问
33   selector:
34     k8s-app: kubernetes-dashboard
35
```

```
36 ---
37 apiVersion: v1
38 kind: Secret
39 metadata:
40   labels:
41     k8s-app: kubernetes-dashboard
42   name: kubernetes-dashboard-certs
43   namespace: kubernetes-dashboard
44 type: Opaque
45
46 ---
47 apiVersion: v1
48 kind: Secret
49 type: Opaque
50 metadata:
51   labels:
52     k8s-app: kubernetes-dashboard
53   name: kubernetes-dashboard-csrf
54   namespace: kubernetes-dashboard
55 data:
56   csrf: ""
57
58 ---
59 apiVersion: v1
60 kind: Secret
61 type: Opaque
62 metadata:
63   labels:
64     k8s-app: kubernetes-dashboard
65   name: kubernetes-dashboard-key-holder
66   namespace: kubernetes-dashboard
67
68 ---
69 kind: ConfigMap
70 apiVersion: v1
71 metadata:
72   labels:
73     k8s-app: kubernetes-dashboard
74   name: kubernetes-dashboard-settings
75   namespace: kubernetes-dashboard
76
77 ---
```

```
78 kind: Role
79 apiVersion: rbac.authorization.k8s.io/v1
80 metadata:
81   labels:
82     k8s-app: kubernetes-dashboard
83   name: kubernetes-dashboard
84   namespace: kubernetes-dashboard
85 rules:
86   - apiGroups: [""]
87     resources: ["secrets"]
88     resourceNames: ["kubernetes-dashboard-key-holder", "kubernetes-dashboard-certs",
89 "kubernetes-dashboard-csrf"]
89     verbs: ["get", "update", "delete"]
90   - apiGroups: [""]
91     resources: ["configmaps"]
92     resourceNames: ["kubernetes-dashboard-settings"]
93     verbs: ["get", "update"]
94   - apiGroups: [""]
95     resources: ["services"]
96     resourceNames: ["heapster", "dashboard-metrics-scraper"]
97     verbs: ["proxy"]
98   - apiGroups: [""]
99     resources: ["services/proxy"]
100    resourceNames: ["heapster", "http:heapster:", "https:heapster:", "dashboard-metrics-
101scraper", "http:dashboard-metrics-scraper"]
101    verbs: ["get"]
102
103 ---
104 kind: ClusterRole
105 apiVersion: rbac.authorization.k8s.io/v1
106 metadata:
107   labels:
108     k8s-app: kubernetes-dashboard
109   name: kubernetes-dashboard
110 rules:
111   - apiGroups: ["metrics.k8s.io"]
112     resources: ["pods", "nodes"]
113     verbs: ["get", "list", "watch"]
114
115 ---
116 apiVersion: rbac.authorization.k8s.io/v1
117 kind: RoleBinding
118 metadata:
```



```
119   labels:
120     k8s-app: kubernetes-dashboard
121   name: kubernetes-dashboard
122   namespace: kubernetes-dashboard
123 roleRef:
124   apiGroup: rbac.authorization.k8s.io
125   kind: Role
126   name: kubernetes-dashboard
127 subjects:
128   - kind: ServiceAccount
129     name: kubernetes-dashboard
130     namespace: kubernetes-dashboard
131
132 ---
133 apiVersion: rbac.authorization.k8s.io/v1
134 kind: ClusterRoleBinding
135 metadata:
136   name: kubernetes-dashboard
137 roleRef:
138   apiGroup: rbac.authorization.k8s.io
139   kind: ClusterRole
140   name: kubernetes-dashboard
141 subjects:
142   - kind: ServiceAccount
143     name: kubernetes-dashboard
144     namespace: kubernetes-dashboard
145
146 ---
147 kind: Deployment
148 apiVersion: apps/v1
149 metadata:
150   labels:
151     k8s-app: kubernetes-dashboard
152   name: kubernetes-dashboard
153   namespace: kubernetes-dashboard
154 spec:
155   replicas: 1
156   revisionHistoryLimit: 10
157   selector:
158     matchLabels:
159       k8s-app: kubernetes-dashboard
160   template:
```

```
161 metadata:
162   labels:
163     k8s-app: kubernetes-dashboard
164 spec:
165   securityContext:
166     seccompProfile:
167       type: RuntimeDefault
168   containers:
169     - name: kubernetes-dashboard
170       # 阿里云镜像，国内极速拉取
171       image: registry.cn-hangzhou.aliyuncs.com/google_containers/dashboard:v2.7.0
172       imagePullPolicy: IfNotPresent
173       ports:
174         - containerPort: 8443
175           protocol: TCP
176       args:
177         - --auto-generate-certificates
178         - --namespace=kubernetes-dashboard
179       volumeMounts:
180         - name: kubernetes-dashboard-certs
181           mountPath: /certs
182         - mountPath: /tmp
183           name: tmp-volume
184       livenessProbe:
185         httpGet:
186           scheme: HTTPS
187           path: /
188           port: 8443
189         initialDelaySeconds: 30
190         timeoutSeconds: 30
191       securityContext:
192         allowPrivilegeEscalation: false
193         readOnlyRootFilesystem: true
194         runAsNonRoot: true
195         runAsUser: 1001
196   volumes:
197     - name: kubernetes-dashboard-certs
198       secret:
199         secretName: kubernetes-dashboard-certs
200     - name: tmp-volume
201       emptyDir: {}
202   serviceAccountName: kubernetes-dashboard
```

```
203     nodeSelector:
204         kubernetes.io/os: linux
205     tolerations:
206         - key: node-role.kubernetes.io/master
207           effect: NoSchedule
208         - key: node-role.kubernetes.io/control-plane
209           effect: NoSchedule
210
211 ---
212 kind: Service
213 apiVersion: v1
214 metadata:
215     labels:
216         k8s-app: dashboard-metrics-scraper
217     name: dashboard-metrics-scraper
218     namespace: kubernetes-dashboard
219 spec:
220     ports:
221         - port: 8000
222           targetPort: 8000
223     selector:
224         k8s-app: dashboard-metrics-scraper
225
226 ---
227 kind: Deployment
228 apiVersion: apps/v1
229 metadata:
230     labels:
231         k8s-app: dashboard-metrics-scraper
232     name: dashboard-metrics-scraper
233     namespace: kubernetes-dashboard
234 spec:
235     replicas: 1
236     revisionHistoryLimit: 10
237     selector:
238         matchLabels:
239             k8s-app: dashboard-metrics-scraper
240     template:
241         metadata:
242             labels:
243                 k8s-app: dashboard-metrics-scraper
244         spec:
```

```
245     securityContext:
246       seccompProfile:
247         type: RuntimeDefault
248     containers:
249       - name: dashboard-metrics-scraper
250         # 阿里云镜像，国内极速拉取
251         image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-
scraper:v1.0.8
252         imagePullPolicy: IfNotPresent
253         ports:
254           - containerPort: 8000
255             protocol: TCP
256         livenessProbe:
257           httpGet:
258             scheme: HTTP
259             path: /
260             port: 8000
261             initialDelaySeconds: 30
262             timeoutSeconds: 30
263         volumeMounts:
264           - mountPath: /tmp
265             name: tmp-volume
266         securityContext:
267           allowPrivilegeEscalation: false
268           readOnlyRootFilesystem: true
269           runAsNonRoot: true
270           runAsUser: 1001
271     serviceAccountName: kubernetes-dashboard
272     nodeSelector:
273       kubernetes.io/os: linux
274     tolerations:
275       - key: node-role.kubernetes.io/master
276         effect: NoSchedule
277       - key: node-role.kubernetes.io/control-plane
278         effect: NoSchedule
279     volumes:
280       - name: tmp-volume
281         emptyDir: {}
```

3. 创建管理员用户并获取令牌

创建一个拥有集群管理员权限的用户，用于登录Dashboard：

```
1 # 创建管理员服务账户
2 kubectl create serviceaccount admin-user -n kubernetes-dashboard
3
4 # 绑定集群管理员权限
5 kubectl create clusterrolebinding admin-user-binding \
6     --clusterrole=cluster-admin \
7     --serviceaccount=kubernetes-dashboard:admin-user
8
9 # 获取登录令牌（有效期1小时）
10 kubectl -n kubernetes-dashboard create token admin-user
```

4. 访问Dashboard

```
1 https://你的服务器IP:30000
```

选择"令牌"登录，粘贴刚才获取的令牌即可。

八、验证部署成功

1. 检查集群组件状态

```
1 kubectl get cs
2 kubectl get pods -n kube-system
```

```
root@k8s-master:~# kubectl get cs
Warning: v1 ComponentStatus is deprecated in v1.19+
NAME                STATUS    MESSAGE             ERROR
controller-manager  Healthy   ok
scheduler            Healthy   ok
etcd-0              Healthy   ok
```

```

root@k8s-master:~# kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
calico-kube-controllers-7c968b5878-8fr45  1/1     Running   0           21h
calico-node-jtnts                      1/1     Running   0           21h
coredns-5dd5756b68-g42ch              1/1     Running   0           21h
coredns-5dd5756b68-tkr2n              1/1     Running   0           21h
etcd-k8s-master                       1/1     Running   0           21h
kube-apiserver-k8s-master              1/1     Running   0           21h
kube-controller-manager-k8s-master     1/1     Running   0           21h
kube-proxy-plgkv                      1/1     Running   0           21h
kube-scheduler-k8s-master              1/1     Running   0           21h

```

所有Pod都应该处于 `Running` 状态。

2. 测试部署一个示例应用

```

1 # 部署Nginx示例
2 kubectl create deployment nginx --image=nginx:1.25
3 kubectl expose deployment nginx --port=80 --type=NodePort
4
5 # 查看NodePort端口
6 kubectl get svc nginx

```

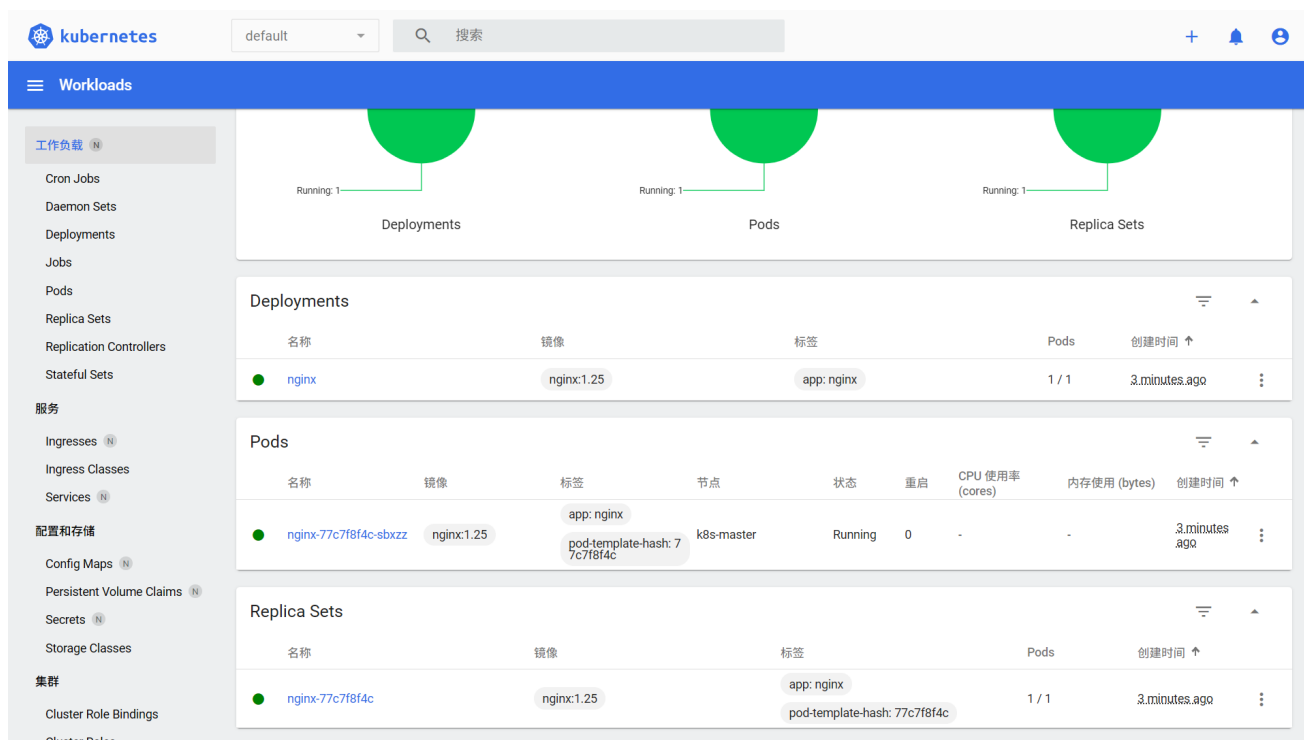
```

root@k8s-master:~# kubectl create deployment nginx --image=nginx:1.25
deployment.apps/nginx created
root@k8s-master:~# kubectl expose deployment nginx --port=80 --type=NodePort
service/nginx exposed
root@k8s-master:~# kubectl get svc nginx
NAME      TYPE      CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
nginx     NodePort  10.99.146.253 <none>         80:32239/TCP    24s

```

然后通过 `http://你的服务器IP:NodePort端口` 访问Nginx页面，验证集群工作正常。

The screenshot shows the Kubernetes dashboard interface. On the left, there is a sidebar with navigation links for Workloads, Services, and Configuration. The main area displays the 'Workload Status' for the 'nginx' deployment. It shows three green circles representing the status of Deployments, Pods, and Replica Sets, all with a 'Running: 1' status. Below this, there are two tables: 'Deployments' and 'Pods'. The 'Deployments' table shows one deployment named 'nginx' with image 'nginx:1.25' and label 'app: nginx', with 1/1 pods and created 2 minutes ago. The 'Pods' table shows one pod named 'nginx-77c7f8f4c-sbxzz' with image 'nginx:1.25' and labels 'app: nginx' and 'pod-template-hash: 77c7f8f4c', running on the 'k8s-master' node with a 'Running' status, 0 restarts, and created 2 minutes ago.



九、常见问题解决

1. kubeadm init失败：检查Swap是否已永久关闭，Containerd配置是否正确，系统参数是否生效。
2. 节点一直NotReady：检查Calico Pod是否正常运行，查看日志 `kubectl logs -f -n kube-system calico-node-xxx`。
3. Dashboard无法访问：确保kubectl proxy正在运行，防火墙已关闭，URL输入正确。
4. 令牌过期：默认令牌有效期为1小时，过期后重新执行 `kubectl -n kubernetes-dashboard create token admin-user` 获取新令牌。
5. Kubernetes Dashboard默认只在集群内部可访问可以通过Dashboard的NodePort暴露方式（无需kubectl proxy，直接通过端口访问）